Desarrollo de un Traductor Espanol-Quechua con RNN

Curso: Inteligencia Artificial - Facultad de Ingenieria y Arquitectura

Programa: Ingenieria de Sistemas - Universidad Andina del Cusco

Repositorio: https://github.com/RedOne003/traductor-quechua-rnn

Demo en linea: https://traductor-quechua-rnn.netlify.app

1. Objetivo

Disenar, implementar y comparar dos traductores automaticos de espanol a quechua basados en arquitecturas de Redes Neuronales Recurrentes (RNN): una RNN simple y una RNN con celdas LSTM, evaluando su desempeno sobre el mismo corpus y las mismas condiciones de entrenamiento.

2. Recopilacion y preprocesamiento de datos

Se utilizo el corpus paralelo publico espanol-quechua (variante Ayacucho, 'quy') del shared task AmericasNLP 2021, derivado de JW300 (Agic & Vulic, 2019; Tiedemann, 2012), disponible en github.com/AmericasNLP/americasnlp2021. El corpus original contiene 125,008 pares de oraciones alineadas.

Preprocesamiento aplicado:

- Normalizacion: minusculas, separacion de signos de puntuacion, limpieza de caracteres atipicos.
- Filtrado por longitud: se conservaron oraciones de 1 a 15 palabras por idioma, para mantener secuencias manejables en una RNN basica.
- Submuestreo a 30,000 pares (de los ~51,600 que superaron el filtro) para que el entrenamiento fuera viable en CPU dentro del tiempo disponible.
- Tokenizacion a nivel de palabra (Keras Tokenizer), vocabulario limitado a 12,000 palabras por idioma, token <unk> para fuera de vocabulario.
- Se agregaron tokens especiales <sos> y <eos> a cada oracion.
- Padding post a longitud fija (17 tokens) con pad_sequences.

Particion final: 29000 pares de entrenamiento, 758 de validacion (dev oficial del shared task) y 1000 de prueba interna (no se usaron para entrenar ni ajustar hiperparametros).

3. Arquitecturas implementadas

Se implemento un esquema encoder-decoder (sequence-to-sequence) identico para ambos modelos, variando unicamente el tipo de celda recurrente, para que la comparacion sea controlada:

- Encoder: capa Embedding (128 dimensiones, con mascara para el padding) seguida de una capa recurrente (SimpleRNN o LSTM, 256 unidades) que entrega el o los estados finales.
- Decoder: Embedding (128) + capa recurrente del mismo tipo, inicializada con el estado del encoder, entrenada con teacher forcing, seguida de una capa densa softmax sobre el vocabulario quechua (12,000 clases).
- Optimizador: Adam. Perdida: sparse_categorical_crossentropy. Metrica: accuracy por token.
- Para la inferencia se construyeron modelos auxiliares de encoder y decoder que permiten decodificacion greedy (paso a paso), reutilizando las mismas capas entrenadas.

4. Entrenamiento

Ambos modelos se entrenaron con el mismo conjunto de datos, mismo tamano de batch (64) y early stopping sobre la perdida de validacion (paciencia de 3 epocas), guardando el mejor checkpoint segun val_loss.

Nota sobre el alcance del entrenamiento: por la restriccion de tiempo de entrega de esta actividad, el entrenamiento se acoto antes de alcanzar convergencia completa (la RNN simple se detuvo por early stopping en la epoca 5; el LSTM se detuvo manualmente en la epoca 3, usando el mejor checkpoint disponible hasta ese punto). Esto se refleja en el desempeno (seccion 6) y se discute como limitacion en la seccion 7.

5. Resultados cuantitativos

Metrica	RNN simple	LSTM
Parametros entrenables	6,353,120	6,944,480
Epocas completadas	5	3
Tiempo de entrenamiento	515 s	532 s
Loss final (train)	3.446	4.041
Loss final (val)	3.933	3.704
Accuracy por token (val)	0.488	0.490
BLEU (test interno, n=40)	0.0111	0.0128

Ejemplos de traduccion

ES: otros también serían ungidos para ser reyes y sacerdotes .

```
REF (quechua):  kanqataqmi pay hina huk reykunapas hinaspa sacerdotekunapas .
RNN simple:     <unk> <unk> <unk> .
LSTM:           chaymi <unk> <unk> <unk> <unk> .
```

ES: queda claro , por tanto , que jesús no es el dios todopoderoso .

```
REF (quechua):  chaynaqa , jesusqa manam tukuy atiyniyoq dioschu .
RNN simple:     <unk> <unk> <unk> .
LSTM:           ichaqa , ¿ imaynatam jehova diosqa <unk> ?
```

ES: un anciano le dijo : te conozco desde que eras un bebé .

```
REF (quechua):  hinaptinmi huk anciano nirqa : ñuqaqa wawacha kasqaykimantam riqsiki .
RNN simple:     <unk> <unk> <unk> .
LSTM:           <unk> <unk> <unk> <unk> .
```

6. Analisis de resultados

El LSTM obtuvo una perdida de validacion menor (3.70 vs 3.93) y un BLEU ligeramente superior (0.0128 vs 0.0111) que la RNN simple, pese a haber completado menos epocas de entrenamiento (3 vs 5). Esto es consistente con la literatura: las celdas LSTM, al contar con una puerta de memoria (cell state) que mitiga el problema del desvanecimiento del gradiente, logran capturar dependencias de mas largo alcance con menos pasos de entrenamiento que una RNN simple.

Los valores de BLEU obtenidos son bajos en terminos absolutos. Esto se debe principalmente a: (1) el entrenamiento se detuvo muy temprano por la restriccion de tiempo disponible para esta entrega, antes de que los modelos convergieran; (2) el quechua es un idioma altamente aglutinante (una sola palabra puede combinar varios morfemas que en espanol se expresan como varias palabras), lo que dificulta la tokenizacion a nivel de palabra y aumenta la dispersion del vocabulario; y (3) el tamano del corpus usado (30,000 pares) es pequeno

para un problema de traduccion automatica neuronal, que tipicamente requiere ordenes de magnitud mas de datos para generalizar bien.

7. RNN simple vs LSTM: ventajas y desventajas

RNN simple:

+ Menos parametros y calculo mas liviano por paso de tiempo.
+ Mas rapida de entrenar por epoca en secuencias cortas.
- Sufre el problema del desvanecimiento/explosion del gradiente en secuencias largas, lo que limita su capacidad de recordar contexto lejano (relevante en traduccion, donde el orden de palabras entre espanol y quechua difiere).
- Tiende a converger a una solucion pobre o estancarse en pocas epocas (se observo early stopping temprano).

LSTM:

+ Las puertas de entrada/olvido/salida permiten retener o descartar informacion de forma selectiva, mitigando el desvanecimiento del gradiente y mejorando el modelado de dependencias largas.
+ En este experimento logro menor loss de validacion y mejor BLEU con menos epocas.
- Mayor costo computacional por paso de tiempo (mas parametros y operaciones) y mayor tiempo de entrenamiento por epoca.
- Requiere mas datos y epocas para explotar completamente su capacidad, por lo que con un entrenamiento muy breve la ventaja relativa frente a la RNN simple, aunque presente, es todavia modesta.

8. Conclusiones

1. Se implemento exitosamente un pipeline completo (descarga de datos, preprocesamiento, entrenamiento, evaluacion e interfaz de demo) para traduccion automatica neuronal espanol-quechua, reutilizable y documentado en el repositorio de GitHub.
2. El LSTM supero a la RNN simple en todas las metricas evaluadas (loss de validacion y BLEU), confirmando la ventaja teorica de las celdas con memoria de largo plazo para tareas de traduccion automatica.
3. El desempeno absoluto de ambos modelos es limitado debido al entrenamiento acotado por tiempo y al tamano reducido del corpus; se recomienda como trabajo futuro entrenar por mas epocas, usar el corpus completo (125k pares), incorporar mecanismos de atencion y tokenizacion subpalabra (BPE) para lidiar con la morfologia aglutinante del quechua.
4. Se desarrollo una interfaz web (Gradio) que permite comparar ambas traducciones en tiempo real, facilitando la demostracion del funcionamiento de ambos traductores.

9. Repositorio y reproducibilidad

Codigo fuente completo, datos de procesamiento, pesos de los modelos y resultados: https://github.com/RedOne003/traductor-quechua-rnn

Demo interactiva desplegada (traductor con vocabulario reducido, 100% en el navegador): https://traductor-quechua-rnn.netlify.app

El README del repositorio detalla los pasos exactos para reproducir el preprocesamiento, el entrenamiento de ambos modelos, la evaluacion BLEU y la demo web.

10. Consideraciones eticas

El corpus utilizado es de acceso publico y se cita a sus autores originales (Agic & Vulic, 2019; Tiedemann, 2012) conforme a los terminos del shared task AmericasNLP. El quechua es una lengua originaria con comunidades de hablantes activas; este proyecto tiene fines estrictamente academicos y no pretende sustituir el trabajo de traductores ni linguistas nativos, sino explorar tecnicas de IA aplicadas a lenguas de bajos recursos, reconociendo las limitaciones del modelo resultante.